

RadiologyAI

High Level Design Document

Architecture, Design Choices & Rationale

V G Masilamani(DA25S005)

Project: Chest X-Ray Disease Classifier

Model: EfficientNetB0 (PyTorch 2.3.0)

Frontend: React.js — 6 pages

Backend: FastAPI — 12 REST endpoints

MLOps: DVC · MLflow · Airflow · Prometheus · Grafana · Evidently AI

Contents

1	System Overview	2
2	Full Component Stack	3
3	Frontend Pages	4
4	DVC Pipeline — 9 Stages	4
5	Airflow DAG — Feedback-Based Retraining	4
6	Monitoring Stack	5
7	Two-Phase Training Strategy	5
8	Design Principles	5
9	Key Design Choices & Rationale	6
10	Security & Ethics	6

System Overview

RadiologyAI is a production-grade AI clinical decision support system that classifies chest X-ray images into Normal, Pneumonia, or COVID-19 using a fine-tuned EfficientNetB0 deep learning model. The system follows a complete MLOps lifecycle and includes real-time monitoring, radiologist feedback loop, automated drift detection, model experiment comparison, and downloadable clinical PDF reports. The system targets clinical operators — ER doctors, radiologists, and hospital screening teams — as a second-opinion decision support tool.

Full Component Stack

Component	Technology	Responsibility
Frontend	React.js (6 pages)	Upload, Results, History, Drift, Model Compare, Monitor
Backend API	FastAPI + Python 3.10	12 REST endpoints — inference, feedback, drift, PDF, MLflow proxy
ML Model	EfficientNetB0 (PyTorch 2.3)	3-class chest X-ray classification
Explainability	Grad-CAM	Heatmap overlay — model attention regions on X-ray
PDF Reports	ReportLab	Downloadable clinical report with Grad-CAM embedded
Experiment Tracking	MLflow 3.x	Hyperparams, metrics, artifacts, model registry
Data Versioning	DVC + Git LFS	9-stage DAG pipeline from download to export
Pipeline Orchestration	Apache Airflow	Feedback-triggered retraining DAG with configurable params
Data Validation	Great Expectations	Schema and quality checks on preprocessed images
Drift Detection	Evidently AI	Pixel distribution drift vs training baseline — HTML reports
Monitoring	Prometheus + Node Exporter	API metrics + system CPU/memory/disk metrics
Visualisation	Grafana (12 panels)	NRT dashboard — latency, error rate, model accuracy, system
Alerting	Alertmanager + Gmail SMTP	Email alerts: backend down, high latency, low accuracy, drift
Containerisation	Docker + Docker Compose	7 independent services
CI/CD	GitHub Actions	Lint + test + DVC check on every push to main
Config Management	Pydantic Settings + .env	All config externalised — zero hardcoded values

Frontend Pages

Page	Nav Label	Description
Analysis	Analysis	Upload X-ray, Analyse, Verdict + Grad-CAM + PDF download
Results	(auto)	Diagnosis card, confidence bars, heatmap, feedback form
History	History	All past scans — filterable table, click to expand detail
Drift Detection	Drift	Evidently AI drift scores per class + HTML report in iframe
Model Comparison	Models	MLflow runs table + bar chart comparison across experiments
Monitor	Monitor	System health, Grafana link, DVC pipeline, MLflow link

DVC Pipeline — 9 Stages

Stage	Script	Input → Output
download	download.py	Kaggle API → data/raw/
validate	validate.py	raw images → validation_report.json + baseline_stats.json
preprocess	preprocess.py	raw → data/processed/ (224×224, 70/15/15 split)
train	train.py	processed/train+val → efficientnetb0_best.pth + metrics
evaluate	evaluate.py	best.pth + test → eval_report.json
explain	gradcam.py	best.pth + test → gradcam_samples/
export	export.py	best.pth → efficientnetb0_torchscript.pt
drift_detection	drift_detection.py	train+test → drift_summary.json + 3 HTML reports
data_validation	great_expectations.py	train → ge_validation_report.json

Airflow DAG — Feedback-Based Retraining

Task	Description
validate_data	Runs validate.py — checks image quality and schema
check_feedback_accuracy	Calls /api/v1/feedback/stats — checks accuracy vs configurable threshold
check_data_drift	Loads baseline_stats.json — checks for pixel distribution drift
branch_retrain_or_skip	Branches: retrain if accuracy below threshold OR drift detected
retrain_model	Runs train.py with DAG-level configurable params (batch, epochs, LR)
evaluate_model	Runs evaluate.py — logs all metrics to MLflow
export_model	Exports to TorchScript — registers in MLflow model registry
notify_completion	Logs pipeline completion with accuracy summary

Configurable DAG Params (set at trigger time via Airflow UI):

- `batch_size` (default: 32, range: 8–128)
- `epochs_phase1` (default: 5, range: 1–20)
- `epochs_phase2` (default: 15, range: 1–50)
- `lr_phase1` (default: 0.001), `lr_phase2` (default: 0.0001)
- `accuracy_threshold` (default: 0.80) — retrain if accuracy below this
- `min_feedback_samples` (default: 10) — minimum samples before retraining
- `force_retrain` (default: false) — bypass all checks and retrain immediately

Monitoring Stack

Tool	Port	What It Monitors
Prometheus	9090	Scrapes backend (:8005/metrics) + <code>node_exporter</code> every 15s
Node Exporter	9100	CPU usage, memory, disk, network — system metrics
Grafana	3001	12-panel dashboard: request rate, latency p50/p95/p99, error rate, model accuracy, feedback rate, CPU, memory, disk, system uptime
Alertmanager	9093	Routes alerts to Gmail SMTP: BackendDown, HighErrorRate, HighLatencyP95, LowModelAccuracy, HighIncorrectPredictions, HighCPUUsage

Two-Phase Training Strategy

Phase	Description
Phase 1 (5 epochs)	Backbone frozen. Only classifier head trains. LR = 0.001. Prevents catastrophic forgetting of ImageNet weights.
Phase 2 (15 epochs)	Full model unfrozen. End-to-end fine-tuning. LR = 0.0001. Achieves higher accuracy.

Design Principles

- **Loose coupling** — React and FastAPI are fully independent. Connected only via configurable REST API. URL set via `REACT_APP_API_URL` environment variable.
- **Config-driven** — All secrets, paths, thresholds loaded from `.env` via Pydantic Settings. Zero hardcoded values in any source file.
- **Reproducibility** — Every training run identified by Git commit hash + MLflow Run ID pair. `dvc repro` guarantees identical results.
- **Explainability** — Grad-CAM mandatory for every prediction. Medical AI requires interpretable results for clinical trust.
- **Feedback loop** — Radiologist feedback drives automated retraining via Airflow DAG. Prometheus gauge updated in real time.
- **Observability** — Prometheus + Grafana + Alertmanager. Alerts fire on threshold breach. Email notifications via Gmail SMTP.
- **Data safety** — X-ray images not stored permanently. Processed for inference only. Feedback stored locally, never shared.

Key Design Choices & Rationale

Decision	Rationale
EfficientNetB0 over ResNet50	5.3M vs 25M parameters. Same accuracy, 3x faster inference. Meets <200ms SLA.
PyTorch over TensorFlow	Better debugging with dynamic computation graphs. Richer torchvision model zoo.
Albumentations augmentation	5x faster than torchvision transforms. GPU-optimised. Richer augmentation options.
Two-phase training	Phase 1 frozen backbone prevents catastrophic forgetting of ImageNet pretrained weights.
Weighted cross-entropy loss	Dataset imbalance (73% Pneumonia). Weighted loss prevents majority-class prediction bias.
DVC for data versioning	5GB+ image datasets cannot go in Git. DVC tracks separately with full Git integration.
Airflow for orchestration	DAG-based pipeline with branching logic. Feedback-triggered retraining. Interactive params.
Evidently AI for drift	Purpose-built for ML data drift. Pixel distribution comparison. Generates HTML reports.
ReportLab for PDF	Pure Python PDF generation. No external service. Grad-CAM image embedded directly.
SQLite for MLflow/Airflow	No cloud restriction per course guidelines. Fully local. Sufficient for academic scale.
Node Exporter	Exposes system CPU, memory, disk metrics to Prometheus. Shows full infrastructure health.

Security & Ethics

- Patient X-ray images are not stored permanently — used for inference only, discarded post-session
- All predictions include a mandatory clinical disclaimer
- Feedback data stored locally — never shared with third parties
- No cloud transmission of medical imaging data — fully on-premise deployment
- Alertmanager uses Gmail App Password — not account password — for SMTP authentication
- All configuration via environment variables — no secrets in source code